

Moving from Notebooks to Scripts

Objectif: How to migrate and organize code from our notebook to Python scripts.

Intuition: Notebooks have been great so far for development. They're interactive, stateful (don't have to rerun code), and allow us to visualize outputs. However, when we want to develop a quality codebase, we need to move to scripts. Here are some reasons why:

- **stateless:** when we run code in a notebook, it's automatically saved to the global state (memory). This is great for experimentation because code and variables will be readily available across different cells. However, this can be very problematic as well because there can be hidden states that we're not aware of. Scripts, on the other hand, are stateless and we have to explicitly pass variables to functions and classes.
- **linear:** in notebooks, the order in which we execute cells matter. This can be problematic because we can easily execute cells out of order. Scripts, on the other hand, are linear and we have to explicitly execute code for each workload.
- **testing:** it's significantly easier to compose and run tests on scripts, as opposed to Jupyter notebooks. This is crucial for ensuring that we have quality code that works as expected.

Setup: We already have all the scripts provided in our repository so let's discuss how this was all organized.

README

It's always a good idea to start organizing our scripts with a `README.md` file. This is where we can organize all the necessary instructions to walk through our codebase. Our README has information on how to set up our environment, how to run our scripts, etc.

The content of the `README.md` file is what everyone will see when they visit your repository on GitHub. So, it's a good idea to keep it updated with the latest information.

Scripts

Let's start by moving our code from notebooks to scripts. We're going to start by creating the different files and directories that we'll need for our project. The exact number and name of these scripts is entirely up to us, however, it's best to organize and choose names that relate to a specific workload. For example, `data.py` will have all our data related functions and classes. And we can also have scripts for configurations (`config.py`), shared utilities (`utils.py`), etc.

```
madewithml/  
├── config.py  
├── data.py  
├── evaluate.py  
├── models.py  
├── predict.py  
├── serve.py  
├── train.py  
├── tune.py  
└── utils.py
```

Functions and classes

Once we have these ready, we can start moving code from our notebooks to the appropriate scripts. It should be intuitive in which script a particular function or class belongs to. If not, we need to rethink the names of our scripts. For example, `train.py` has functions from our notebook such as `train_step`, `val_step`, `train_loop_per_worker`, etc.

```
# train.py  
def train_step(...):  
    pass  
  
def val_step(...):  
    pass  
  
def train_loop_per_worker(...):  
    pass  
  
...
```

Workloads: Recall that for training a model, we wrote code in our notebook for setting configurations, training, etc. that was freeform in a code cell:

```

# Scaling config
scaling_config = ScalingConfig(
    num_workers=num_workers,
    use_gpu=bool(resources_per_worker["GPU"]),
    resources_per_worker=resources_per_worker,
    _max_cpu_fraction_per_node=0.8,
)

# Checkpoint config
checkpoint_config = CheckpointConfig(
    num_to_keep=1,
    checkpoint_score_attribute="val_loss",
    checkpoint_score_order="min",
)

...

```

These code cells are not part of a function or class, so we need to wrap them around a function so that we can easily execute that workload. For example, all this training logic is wrapped inside a `train_model` function in `train.py` that has all the required inputs to execute the workload:

```

# train.py
def train_model(experiment_name, dataset_loc, ...):
    ...

    # Scaling config
    scaling_config = ScalingConfig(
        num_workers=num_workers,
        use_gpu=bool(gpu_per_worker),
        resources_per_worker={"CPU": cpu_per_worker, "GPU": gpu_per_worker},
        _max_cpu_fraction_per_node=0.8,
    )

    # Checkpoint config
    checkpoint_config = CheckpointConfig(
        num_to_keep=1,
        checkpoint_score_attribute="val_loss",
        checkpoint_score_order="min",
    )

    ...

```

Config: In addition to our core workload scripts, recall that we also have a `config.py` script. This file will include all the setup and configuration that all/most of our workloads depend on. For example, setting up our model registry:

```
# Config MLflow
MODEL_REGISTRY = Path("/tmp/mlflow")
Path(MODEL_REGISTRY).mkdir(parents=True, exist_ok=True)
MLFLOW_TRACKING_URI = "file://" + str(MODEL_REGISTRY.absolute())
mlflow.set_tracking_uri(MLFLOW_TRACKING_URI)
```

We wouldn't have configurations like our `ScalingConfig` here because that's specific to our training workload. The `config.py` script is for configurations that are shared across different workloads.

Utilities: Similarly, we also have a `utils.py` script to include components that will be reused across different scripts. It's a good idea to organize these shared components here as opposed to the core scripts to avoid circular dependency conflicts (two scripts call on functions from each other). Here is an example of one of our utility functions, `set_seeds`, that's used in both our `train.py` and `tune.py` scripts.

```
def set_seeds(seed: int = 42):
    """Set seeds for reproducibility."""
    np.random.seed(seed)
    random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed(seed)
    eval("setattr(torch.backends.cudnn, 'deterministic', True)")
    eval("setattr(torch.backends.cudnn, 'benchmark', False)")
    os.environ["PYTHONHASHSEED"] = str(seed)
```

Ray: Recall in our setup lesson that we initialized Ray inside our notebooks. We still need to initialize Ray before executing our ML workloads via scripts, but we can decide to do this only for the scripts with Ray dependent workloads. For example, at the bottom of our `train.py` script, we have:

```
# /train.py
if __name__ == "__main__":
    if ray.is_initialized():
        ray.shutdown()
    ray.init()
    app() # initialize Typer app
```